

Data Structures and Algorithms: Assignment 2
--

1. List Abstract Data Type (ADT) and Linked List.

Solution:

```
#include <iostream>
using namespace std;

template <typename T>
class LinkedList {
private:
    struct Node {
        T obj;
        Node* next = nullptr;
        Node(const T& x) : obj(x) {}
    };
    Node* root = nullptr;
    Node* findAt(int pos) {
        Node* current_node = root;
        for (int i = 0; i < pos; i++) {
            if (current_node == nullptr) {
                return nullptr;
            }
            current_node = current_node->next;
        }
        return current_node;
    };

public:
    bool insertAt(const T& obj, int pos) {
        if (pos == 0) {
            Node* node = new Node(obj);
            node->next = root;
            root = node;
            return true;
        } else {
            Node* prev = findAt(pos - 1);
            if (prev == nullptr) {
                return false;
            } else {
                Node* node = new Node(obj);
                node->next = prev->next;
                prev->next = node;
                return true;
            }
        }
    };

    bool insertFirst(const T& obj) {
        return insertAt(obj, 0);
    };

    bool insertLast(const T& obj) {
        Node* curr = root;
        if (root == nullptr) {
            return insertFirst(obj);
        }
        Node* node = new Node(obj);
        while (curr->next != nullptr) {
            curr = curr->next;
        }
        curr->next = node;
    };
};
```

```
        return true;
    };

    bool deleteAt(int pos) {
        if (pos == 0) {
            return deleteFirst();
        }
        Node* prev = findAt(pos - 1);
        if (prev == nullptr) {
            return false;
        }
        Node* curr = prev->next;

        if (curr == nullptr) {
            return false;
        }
        prev->next = curr->next;
        delete curr;
        return true;
    };

    bool deleteFirst() {
        Node* node = root;
        if (node == nullptr) {
            return false;
        }
        root = node->next;
        delete node;
        return true;
    }

    bool deleteLast() {
        if (root == nullptr) {
            return false;
        }
        Node* curr = root;
        if (root->next == nullptr) {
            root = nullptr;
            delete curr;
            return true;
        }
        Node* prev = nullptr;
        while (curr->next != nullptr) {
            prev = curr;
            curr = curr->next;
        }
        prev->next = nullptr;
        delete curr;
        return true;
    };

    bool inList(const T& obj) const {
        Node* curr = root;
        while (curr != nullptr) {
            if (curr->obj == obj) {
                return true;
            }
            curr = curr->next;
        }
        return false;
    };

    void printList() const {
        Node* curr = root;
        while (curr != nullptr) {
            cout << curr->obj;
```

```
        cout << endl;
        curr = curr->next;
    }
};

int size() const {
    int cnt = 0;
    Node* curr = root;
    while (curr != nullptr) {
        cnt++;
        curr = curr->next;
    }
    return cnt;
}

void reverse() {
    Node* prev = nullptr;
    Node* curr = root;
    Node* next = nullptr;
    while (curr != nullptr) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    root = prev;
};

};

int main() {
    LinkedList<int> intList;
    intList.insertFirst(1);
    intList.insertLast(3);
    intList.insertAt(2, 1);
    intList.printList();
    cout << endl;
    cout << endl;
    for (int i = 4; i <= 10; i++) {
        intList.insertLast(i);
    }

    intList.printList();
    cout << endl << endl;
    cout << "5 in list?" << endl << intList.inList(5) << endl << endl;
    cout << "Delete at 5? " << intList.deleteAt(4) << endl << endl;
    cout << "5 in list?" << endl << intList.inList(5) << endl << endl;

    cout << "10 in list?" << endl << intList.inList(10) << endl << endl;
    cout << "Delete last? " << intList.deleteLast() << endl << endl;
    cout << "10 in list?" << endl << intList.inList(10) << endl << endl;
    cout << "1 in list?" << endl << intList.inList(1) << endl << endl;
    cout << "Delete first? " << intList.deleteFirst() << endl << endl;
    cout << "1 in list?" << endl << intList.inList(1) << endl << endl;

    bool run = true;
    while (run) {
        intList.printList();
        cout << endl;
        run = intList.deleteFirst();
    }
    cout << intList.size();
    cout << endl;
    intList.printList();
}
```

```
LinkedList<string> stringList;

stringList.insertFirst("a");
stringList.insertFirst("b");
stringList.insertFirst("c");
stringList.insertFirst("d");
stringList.insertFirst("e");

cout << endl;
stringList.printList();
stringList.reverse();
cout << endl;
stringList.printList();
cout << endl;

run = true;
while (run) {
    stringList.printList();
    cout << endl;
    run = stringList.deleteFirst();
}
cout << stringList.size();
cout << endl;

stringList.insertLast("a");
stringList.reverse();
stringList.insertLast("b");

cout << endl;
stringList.printList();
stringList.reverse();
cout << endl;
stringList.printList();
stringList.insertFirst("c");
cout << endl;
stringList.printList();
stringList.reverse();
cout << endl;
stringList.printList();

return 0;
}
```

Here is the output of the tests:

```
1
2
3

1
2
3
4
5
6
7
8
9
10

5 in list?
```

```
1
Delete at 5? 1
5 in list?
0
10 in list?
1
Delete last? 1
10 in list?
0
1 in list?
1
Delete first? 1
1 in list?
0
2
3
4
6
7
8
9
3
4
6
7
8
9
4
6
7
8
9
6
7
8
9
7
8
9
8
9
9
0
e
d
c
```

b	
a	
a	
b	
c	
d	
e	
a	
b	
c	
d	
e	
b	
c	
d	
e	
c	
d	
e	
d	
e	
e	
0	
a	
b	
b	
a	
c	
b	
a	
a	
b	
c	

2. Implement a stack.

Solution:

```

#include <iostream>
using namespace std;

template <typename T>
class Stack {
private:
    T* data;
    void grow() {
        T* newData = new T[2 * cap];
        for (int i = 0; i < cap; i++) { // THIS IS AN O(N) OPERATION
            newData[i] = data[i];
        }
        delete[] data;
        data = newData;
        cap = 2 * cap;
        return;
    }
    int cap;
    int numElems;
public:
    Stack(int x = 100) {
        cap = x;
        numElems = 0;
        data = new T[cap];
    }
    void push(T val) { // Amortized O(N)
        if (numElems == cap) { // Only O(N) if this is true
            grow();
        }
        data[numElems] = val;
        numElems++;
    }
    T pop() { // O(1)
        numElems--;
        return data[numElems];
    }
    T top() { // O(1)
        return data[numElems - 1];
    }
    int size() {
        return numElems;
    }
    int capacity() {
        return cap;
    }
};

int main() {
    Stack<int> stack(16);

    cout << "Stack capacity: " << stack.capacity() << endl;
    cout << "Stack occupancy: " << stack.size() << endl << endl;
    for (int i = 0; i < 34; i++) {
        stack.push(i);
        cout << "Stack capacity: " << stack.capacity() << endl;
        cout << "Stack occupancy: " << stack.size() << endl << endl;
    }
}

```

Pushing is an amortized $O(N)$. It takes $O(1)$ to push if we do not need to resize, and $O(N)$ if we do need to resize.

Pop and top are $O(1)$, as we just index into the array.

Output from the main test:

```
Stack capacity: 16
Stack occupancy: 0

Stack capacity: 16
Stack occupancy: 1

Stack capacity: 16
Stack occupancy: 2

Stack capacity: 16
Stack occupancy: 3

Stack capacity: 16
Stack occupancy: 4

Stack capacity: 16
Stack occupancy: 5

Stack capacity: 16
Stack occupancy: 6

Stack capacity: 16
Stack occupancy: 7

Stack capacity: 16
Stack occupancy: 8

Stack capacity: 16
Stack occupancy: 9

Stack capacity: 16
Stack occupancy: 10

Stack capacity: 16
Stack occupancy: 11

Stack capacity: 16
Stack occupancy: 12

Stack capacity: 16
Stack occupancy: 13

Stack capacity: 16
Stack occupancy: 14

Stack capacity: 16
Stack occupancy: 15

Stack capacity: 16
Stack occupancy: 16

Stack capacity: 32
Stack occupancy: 17

Stack capacity: 32
Stack occupancy: 18

Stack capacity: 32
Stack occupancy: 19

Stack capacity: 32
Stack occupancy: 20
```

```
Stack capacity: 32  
Stack occupancy: 21
```

```
Stack capacity: 32  
Stack occupancy: 22
```

```
Stack capacity: 32  
Stack occupancy: 23
```

```
Stack capacity: 32  
Stack occupancy: 24
```

```
Stack capacity: 32  
Stack occupancy: 25
```

```
Stack capacity: 32  
Stack occupancy: 26
```

```
Stack capacity: 32  
Stack occupancy: 27
```

```
Stack capacity: 32  
Stack occupancy: 28
```

```
Stack capacity: 32  
Stack occupancy: 29
```

```
Stack capacity: 32  
Stack occupancy: 30
```

```
Stack capacity: 32  
Stack occupancy: 31
```

```
Stack capacity: 32  
Stack occupancy: 32
```

```
Stack capacity: 64  
Stack occupancy: 33
```

```
Stack capacity: 64  
Stack occupancy: 34
```

3. Write an implementation of Queue using Stack(s).

Solution:

```

#include <iostream>
using namespace std;

template <typename T>
class Stack {
private:
    T* data;
    void grow() {
        T* newData = new T[2 * cap];
        for (int i = 0; i < cap; i++) {
            newData[i] = data[i];
        }
        delete[] data;
        data = newData;
        cap = 2 * cap;
        return;
    }
    int cap;
    int numElems;
public:
    Stack(int x = 100) {
        cap = x;
        numElems = 0;
        data = new T[cap];
    }
    void push(T val) {
        if (numElems == cap) {
            grow();
        }
        data[numElems] = val;
        numElems++;
    }
    T pop() {
        numElems--;
        return data[numElems];
    }
    T top() {
        return data[numElems - 1];
    }
    bool isEmpty() const {
        return (numElems == 0);
    }
};

template <typename T>
class Queue {
private:
    Stack<T> stack;
    Stack<T> revStack;
public:
    void push(T val) {
        stack.push(val);
        return;
    }
    T pop() {
        while (not stack.isEmpty()) {
            revStack.push(stack.pop());
        }
        T val = revStack.pop();
        while (not revStack.isEmpty()) {

```

```
        stack.push(revStack.pop());
    }
    return val;
}
T front() {
    while (not stack.isEmpty()) {
        revStack.push(stack.pop());
    }
    T val = revStack.top();
    while (not revStack.isEmpty()) {
        stack.push(revStack.pop());
    }
    return val;
}
};
int main() {
    Queue<int> queue;
    for (int i = 0; i < 18; i++) {
        queue.push(i);
    }
    for (int i = 0; i < 18; i++) {
        cout << "Pop item: " << queue.pop() << endl << endl;
    }
}
```

Output from the main test:

```
Pop item: 0
Pop item: 1
Pop item: 2
Pop item: 3
Pop item: 4
Pop item: 5
Pop item: 6
Pop item: 7
Pop item: 8
Pop item: 9
Pop item: 10
Pop item: 11
Pop item: 12
Pop item: 13
Pop item: 14
Pop item: 15
Pop item: 16
Pop item: 17
```

4. Hash-table using $h(x) = x \bmod 7$.**Solution:** Part a:

0	28
1	15
2	
3	17, 10
4	
5	5, 19, 33, 12
6	20

With a load factor of $9/7$

Part b:

0	28
1	15
2	20
3	33
4	12
5	5
6	19

17 and 10 fail to place, as there is no room. Load factor is $7/7$.

Part c:

0	28
1	15
2	33
3	20
4	17
5	5
6	19

While slot 4 is free when 12 is entered, it cannot find it, and thus it fails to place. 10 fails to place as there is no room.

5. Implement a Set using either a Queue, List or Stack

Solution:

```
#include <iostream>
using namespace std;

template <typename T>
class LinkedList {
private:
    struct Node {
        T obj;
        Node* next = nullptr;
        Node(const T& x) : obj(x) {}
    };
    Node* root = nullptr;
    Node* findAt(int pos) {
        Node* current_node = root;
        for (int i = 0; i < pos; i++) {
            if (current_node == nullptr) {
                return nullptr;
            }
            current_node = current_node->next;
        }
        return current_node;
    };

public:
    T getAt(int pos) {
        return findAt(pos)->obj;
    }
    bool insertAt(const T& obj, int pos) {
        if (pos == 0) {
            Node* node = new Node(obj);
            node->next = root;
            root = node;
            return true;
        } else {
            Node* prev = findAt(pos - 1);
            if (prev == nullptr) {
                return false;
            } else {
                Node* node = new Node(obj);
                node->next = prev->next;
                prev->next = node;
                return true;
            }
        }
    };

    bool insertFirst(const T& obj) {
        return insertAt(obj, 0);
    };

    bool insertLast(const T& obj) {
        Node* curr = root;
        if (root == nullptr) {
            return insertFirst(obj);
        }
        Node* node = new Node(obj);
        while (curr->next != nullptr) {
            curr = curr->next;
        }
        curr->next = node;
        return true;
    };
};
```

```
};  
bool deleteAt(int pos) {  
    if (pos == 0) {  
        return deleteFirst();  
    }  
    Node* prev = findAt(pos - 1);  
    if (prev == nullptr) {  
        return false;  
    }  
    Node* curr = prev->next;  
  
    if (curr == nullptr) {  
        return false;  
    }  
    prev->next = curr->next;  
    delete curr;  
    return true;  
};  
bool deleteFirst() {  
    Node* node = root;  
    if (node == nullptr) {  
        return false;  
    }  
    root = node->next;  
    delete node;  
    return true;  
}  
bool deleteLast() {  
    if (root == nullptr) {  
        return false;  
    }  
    Node* curr = root;  
    if (root->next == nullptr) {  
        root = nullptr;  
        delete curr;  
        return true;  
    }  
    Node* prev = nullptr;  
    while (curr->next != nullptr) {  
        prev = curr;  
        curr = curr->next;  
    }  
    prev->next = nullptr;  
    delete curr;  
    return true;  
};  
  
bool inList(const T& obj) const {  
    Node* curr = root;  
    while (curr != nullptr) {  
        if (curr->obj == obj) {  
            return true;  
        }  
        curr = curr->next;  
    }  
    return false;  
};  
  
void printList() const {  
    Node* curr = root;  
    while (curr != nullptr) {  
        cout << curr->obj;  
        cout << endl;  
    }  
}
```

```

        curr = curr->next;
    }
};

int size() const {
    int cnt = 0;
    Node* curr = root;
    while (curr != nullptr) {
        cnt++;
        curr = curr->next;
    }
    return cnt;
}

void reverse() {
    Node* prev = nullptr;
    Node* curr = root;
    Node* next = nullptr;
    while (curr != nullptr) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    root = prev;
};
};

template <typename T>
class Set {
private:
    LinkedList<T> data;
public:
    bool insert(T val) {
        if (data.inList(val)) {
            return false;
        }
        return data.insertLast(val);
    };
    bool contains(T val) {
        return data.inList(val);
    };
    bool remove(T val) {
        if (data.inList(val)) {
            int N = data.size();
            for (int i = 0; i < N; i++) {
                if (data.getAt(i) == val) {
                    data.deleteAt(i);
                    return true;
                }
            }
        }
        return false;
    };
    void printSet() {
        data.printList();
    }
};

int main() {
    Set<int> set;

    for (int i = 0; i < 8; i++) {

```

```
        set.insert(i);
    }
    set.printSet();
    set.remove(5);
    set.remove(0);
    cout << endl;
    set.printSet();

    set.insert(1);
    cout << endl;
    set.printSet();
}
```

Implemented using the linkedlist from question 1. The output of the tests is:

```
0
1
2
3
4
5
6
7

1
2
3
4
6
7

1
2
3
4
6
7
```

6. Implement a Dictionary

Solution:

```

#include <iostream>
#include <vector>
#include <utility>
using namespace std;

template <typename K, typename V>
class Dictionary {
private:
    vector<pair<K, V>> data;
public:
    bool insert(const K& key, const V& value) {
        if (contains(key)) {
            return false;
        }
        data.push_back({ key, value });
        return true;
    };
    bool contains(const K& key) const {
        for (auto& p : data) {
            if (p.first == key) {
                return true;
            }
        }
        return false;
    };
    bool remove(const K& key) {
        for (int i = 0; i < data.size(); i++) {
            if (data[i].first == key) {
                data.erase(data.begin() + i);
                return true;
            }
        }
        return false;
    };
    V get(const K& key) const {
        for (auto& p : data) {
            if (p.first == key) {
                return p.second;
            }
        }
    };
};

int main() {
    Dictionary<int, string> dict;
    dict.insert(1, "a");
    dict.insert(55, "AAAA");

    cout << "Dict contains 1? " << dict.contains(1) << endl << endl;
    cout << "What does 1 contain? " << dict.get(1) << endl << endl;
    cout << "Has 1 been removed? " << dict.remove(1) << endl << endl;
    cout << "Dict contains 1? " << dict.contains(1) << endl << endl;

    cout << "Dict contains 55? " << dict.contains(55) << endl << endl;
    cout << "What does 55 contain? " << dict.get(55) << endl << endl;
    cout << "Has 55 been removed? " << dict.remove(55) << endl << endl;
    cout << "Dict contains 55? " << dict.contains(55) << endl << endl;
}

```

Output of the tests:

```
Dict contains 1? 1
```

```
What does 1 contain? a
```

```
Has 1 been removed? 1
```

```
Dict contains 1? 0
```

```
Dict contains 55? 1
```

```
What does 55 contain? AAAA
```

```
Has 55 been removed? 1
```

```
Dict contains 55? 0
```

7. An old exam question

Solution: First we check the table.

$22 \bmod 11 = 0$, which fits.

$5 \bmod 11 = 5$, which fits.

$16 \bmod 11 = 5$, collision, $+1$ gives 6, which fits.

$27 \bmod 11 = 5$, collision, $+1$ gives 6, collision, $+4$ gives 9, which fits.

The table is correct.

Adding 1:

$1 \bmod 11 = 1$, fits.

Adding 12:

$12 \bmod 11 = 1$, collision, $+1$ gives 2, fits.

Adding age, 29:

$29 \bmod 11 = 7$, fits

Adding student ID, 201708076:

$201708076 \bmod 11 = 9$, collision, $+1$ gives 10, fits

Final table:

0	22
1	1
2	12
3	
4	
5	5
6	16
7	29
8	
9	27
10	201708076